LLDD BE - API Document Create and Update

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน
และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	32 ชั่วโมง = implementation 24 + unit test 8 (30%)
Owner	Butsaba <But> Podamrong
Target repository	`SBP/srm-sps-spsap-store-backend` (NestJS + TypeORM · schema `sps_store`) + `SBP/srm-sps-spsap-sbp-bff` (forward ผ่าน client service · ไม่มี DB) สำหรับเส้นที่ FE เรียก
Objective	ออกแบบ APIs สำหรับสร้างเอกสารใหม่และบันทึกส่วนย่อยของเอกสาร

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ถูกเรียกจาก	LLDD-FE-Create-Document.pdf	`POST /api/v1/sgi/document`
ถูกเรียกจาก	LLDD-FE-Document-Detail.pdf	`PUT /api/v1/sgi/document/{docNo}`
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
workflow engine	LLDD-BE-Workflow-Engine-Definition.pdf	นิยาม state/route/event ที่หัวข้อ Workflow Trigger Event Contract เรียกใช้
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Database-Structure.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

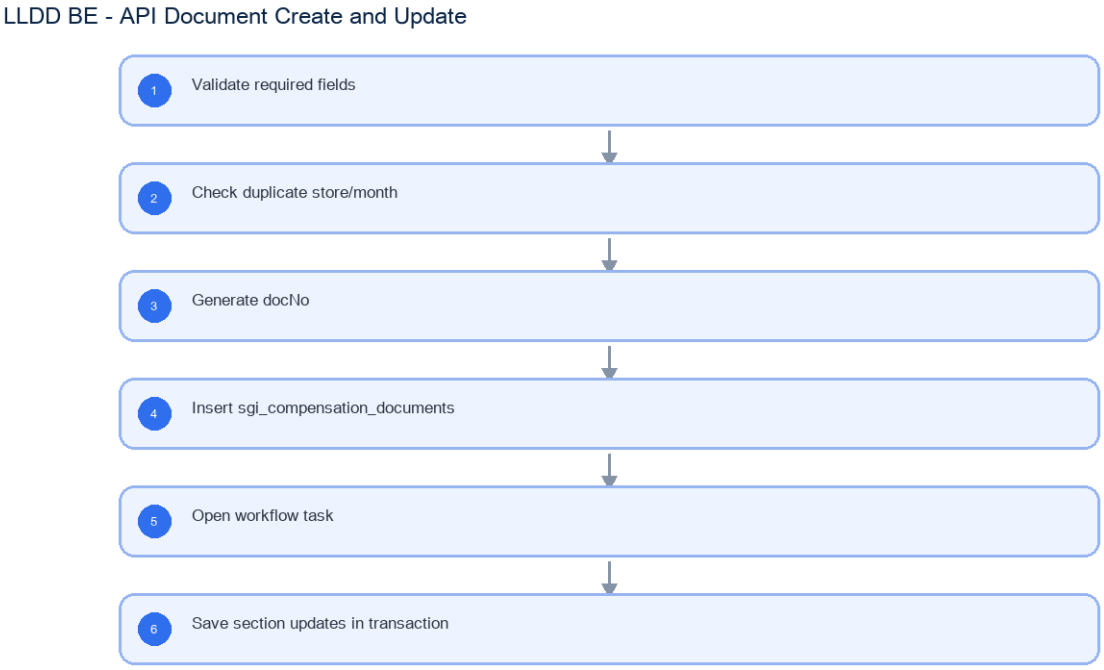
- Create document
- Duplicate guard
- Running doc number
- Partial update
- Business validation

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

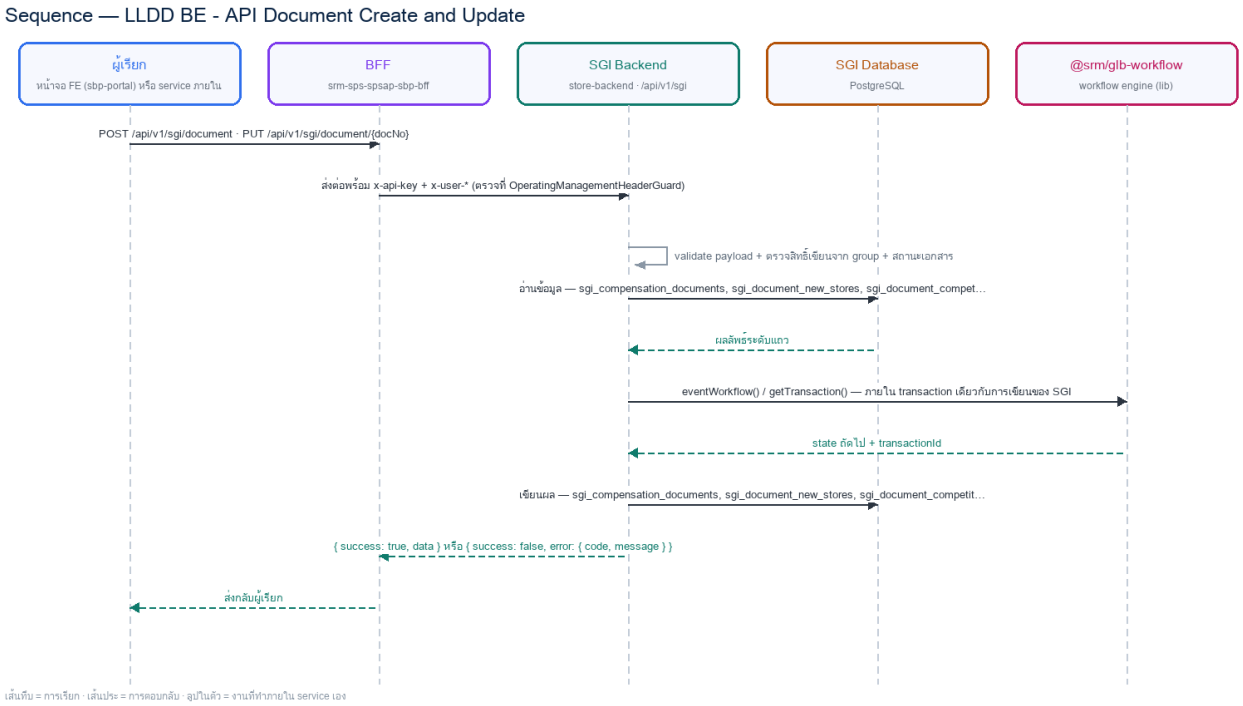
4.1 Implementation Flow (ลำดับขั้นการทำงาน)



รูปที่ 1: Implementation flow reference: LLDD BE - API Document Create and Update

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ



รูปที่ 2: Sequence diagram: LLDD BE - API Document Create and Update

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
docNo	YYYY/xxxxx	required when opening existing document	ใช้ปี ค.ศ. และ running 5 หลัก (มติ 2026-08-06)
storeCode	string 5 digits	numeric length = 5	แสดง leading zero
amount	number, 2 decimals	>= 0	format `#,##0.00` บาท
percent	number, 2 decimals	0-100	ใช้ % และรวม allocation ต้องเท่ากับ 100 — B5: เพิ่ม/ลบร้านที่กระทบเพิ่มเมื่อไร ต้องเกลี่ยใหม่ทั้งชุดแล้วคำนวณ `compensateAmount` ของทุกแถวใหม่ ไม่ใช่เฉพาะแถวที่เพิ่ม
sourceSystem	enum	ALLMAP / USER	B5 ที่มาของแถวร้านเปิดใหม่ — `ALLMAP` ระบบ default ให้อัตโนมัติ (Job 9) · `USER` เจ้าหน้าที่ SBP DSA คีย์เองจากเอกสารแจ้งของหน่วยงานส่งเสริม (ผัง To-Be · SDD สไลด์ 7) · ซ้ำ (`doc_no`, `new_store_code`) ให้คืน `409`
date	DD/MM/YYYY	valid date	payload เป็น ISO ค.ศ. · FE แสดง ค.ศ. เป็นค่าเริ่มต้น (DatePicker buddhistEra=false) แสดง พ.ศ. เฉพาะจุดที่เปิด flag
attachment	file	<= 5 MB	รองรับ vsd, dwg, afp, pdf, mda, zip, wav, mp3, gif, jpg, tif, tiff, htm, html, txt, xml, mpg, mov, ivs, doc, docx, xls, xlsx, pps, ppt, pot, csv
requestId	string	optional	ใช้ trace request; duplicate guard หลักเป็น business key
source	MANUAL|FS	required	แยกแหล่งสร้างเอกสาร

5.1 docNo Generator and Concurrency Rules

เลขเอกสารเป็น business identifier ของระบบ จึงต้อง generate ฝั่ง BE ใน transaction เดียวกับการสร้างเอกสาร และต้องไม่ให้ FE หรือ Job สร้างเลขเอง

Rule	Required behavior	Implementation note
Format	YYYY/xxxxx โดย YYYY เป็นปี ค.ศ. และ running 5 หลัก (มติ 2026-08-06 · หน้าจอ K2 จริงใช้ ค.ศ. เช่น 2026/01870)	ตัวอย่าง 2026/00124; เก็บ doc_no เป็น string และเก็บ year/running_no แยกเพื่อ index
Sequence scope	running reset ตามปี ค.ศ.	unique key (`year, running_no`) และ unique `doc_no`
Lock strategy	lock row sequence ด้วย `SELECT ... FOR UPDATE` หรือ database sequence ต่อปี	ห้ามอ่าน max(running_no)+1 แบบไม่มี lock
Transaction boundary	generate docNo, insert sgi_compensation_documents, insert first workflow task และ audit ใน transaction เดียว	ถ้าสร้าง task ไม่สำเร็จต้อง rollback ทั้งชุด
Gap policy	เลขที่ถูก commit แล้วห้าม reuse; rollback ก่อน commit ไม่ควรเผยแพร่ docNo ให้ client	ถ้าใช้ native sequence ที่เกิด gap ได้ต้องบันทึก policy นี้ใน runbook
Duplicate guard	business key ซ้ำต้องคืน 409 ก่อน generate docNo ใหม่เมื่อเป็นไปได้	business key อย่างน้อย impactedStoreCode+impactMonth+newStoreCode+roundNo+source
Idempotency	requestId ใช้ trace/retry แต่ไม่แทน duplicate business key	ถ้า retry request เดิมหลัง success ให้คืน docNo เดิมเมื่อจับคู่ requestId ได้

5.2 Create Document Transaction Flow

Step	Service behavior	Rollback / error rule
1. Validate input	ตรวจ required, format, store exists, period, source, roundNo	invalid คืน 400/422 ก่อน lock sequence
2. Check duplicate	query business key บน sgi_compensation_documents	พบเอกสารเดิมคืน 409 DUPLICATE_DOCUMENT พร้อม docNo เดิมถ้าอนุญาตให้แสดง
3. Start transaction	เปิด transaction และ lock sequence row ของปี ค.ศ.	lock timeout คืน 409/503 ตามมาตรฐาน platform
4. Generate docNo	เพิ่ม running_no และประกอบ doc_no	ยังไม่ส่ง response จนกว่า commit สำเร็จ
5. Insert document	insert sgi_compensation_documents และ child rows เริ่มต้น	fail ต้อง rollback sequence/document
6. Open first task	เรียก initializeWorkflow + addPreApprover (state 06) ของ @srm/glb-workflow ภายใน transaction boundary ที่กำหนด — ชื่อ function ตามชีต Detail ของ LLDD lib — ดู LLDD-BE-Workflow-Engine-Definition.pdf 5.3	fail ต้อง rollback document
7. Commit	commit transaction (ไม่มีการเขียน audit ของ master แล้ว · ยกเลิกระบบ audit ของ master 2026-08-07)	หลัง commit จึง return docNo/statusCode

5.3 Required Developer Tests for docNo

Test	Expected result
ยิง POST /sgi/document พร้อมกัน 20 request ในปีเดียวกัน	ได้ docNo ไม่ซ้ำ running เรียงตาม commit และไม่มี duplicate key error ที่หลุดเป็น 500
สร้าง duplicate business key	คืน 409 DUPLICATE_DOCUMENT และไม่ consume docNo ใหม่ถ้า duplicate ถูกพบก่อน lock sequence
จำลอง error หลัง insert document ก่อนเปิด workflow	rollback แล้วไม่เหลือ sgi_compensation_documents/workflow_transaction/audit partial
เปลี่ยนปี ค.ศ.	running เริ่มที่ 00001 ของปีใหม่

5.4 docNo Generator SQL Reference

```
-- bind ตามลำดับ: $1=year · $2=docNo · $3=runningNo · $4=impactProcessId · $5=impactCompensationId · $6=impactedStoreCode ·
$7=impactMonth · $8=newStoreCode · $9=roundNo · $10=source · $11=statusInit · $12=userId
-- ออกเลขเอกสาร YYYY/xxxxx แบบ atomic ต่อ "ปี ค.ศ." (ห้ามใช้ พ.ศ. — ดู API design มติ 2026-08-06)
-- ตารางจริงคือ sgi_document_running_numbers (year · last_running_no · updated_at) ไม่มีคอลัมน์ created_at

-- 1) สร้างแถวของปีนี้ถ้ายังไม่มี (idempotent)
INSERT INTO sgi_document_running_numbers (year, last_running_no)
VALUES ($1 /* year */, 0)
ON CONFLICT (year) DO NOTHING;

-- 2) กินเลขถัดไปในทรานแซกชันเดียวกับการสร้างเอกสาร — UPDATE ... RETURNING ล็อกแถวให้เอง
-- กัน batch (Job 8) กับผู้ใช้สร้างพร้อมกันแล้วได้เลขชนกัน
UPDATE sgi_document_running_numbers
SET last_running_no = last_running_no + 1,
    updated_at = CURRENT_TIMESTAMP
WHERE year = $1 /* year */
RETURNING last_running_no; -- → :runningNo

-- 3) docNo = :year || '/' || lpad(:runningNo::text, 5, '0') เช่น 2026/00123
-- ▲ ต้องใส่ impact_process_id ทุกครั้ง — เป็น NOT NULL UNIQUE (หนึ่ง impact process = หนึ่งเอกสาร)
INSERT INTO sgi_compensation_documents (
    doc_no, year, running_no,
    impact_process_id, impact_compensation_id, impacted_store_code, impact_month, new_store_code,
    round_no, source, status_code, current_section_code, created_by
) VALUES (
    $2 /* docNo */, $1 /* year */, $3 /* runningNo */,
    $4 /* impactProcessId */, $5 /* impactCompensationId */, $6 /* impactedStoreCode */, $7 /* impactMonth */, $8 /* newStoreCode */,
    $9 /* roundNo */, $10 /* source */, $11 /* statusInit */, '06', $12 /* userId */
);
-- created_at / total_compensation_amount / version_no มี DEFAULT อยู่แล้ว ไม่ต้องส่ง
```

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	POST /api/v1/sgi/document; PUT /api/v1/sgi/document/{docNo}
Progress	Validate required fields; Check duplicate store/month; Generate docNo; Insert sgi_compensation_documents
Output	sgi_compensation_documents; sgi_document_new_stores; sgi_document_competitors

5.90 Endpoint Implementation Contract

Endpoint	Use-case owner	Service/repository behavior	Definition of done
POST /api/v1/sgi/document	Create document API	Validate required fields	duplicate business key returns 409
PUT /api/v1/sgi/document/{docNo}	Update document partial sections	Check duplicate store/month	docNo format YYYY/xxxxx

5.91 Backend Execution Sequence

Step	Behavior specific to this LLDD	Failure/test evidence
1	Validate required fields	create success
2	Check duplicate store/month	create duplicate
3	Generate docNo	update allocation invalid
4	Insert sgi_compensation_documents	permission denied section
5	Open workflow task	— (ยังไม่มี test เฉพาะขั้นนี้ · ครอบด้วย test รวมของเอกสารในหัวข้อ 11)
6	Save section updates in transaction	— (ยังไม่มี test เฉพาะขั้นนี้ · ครอบด้วย test รวมของเอกสารในหัวข้อ 11)

5.92 Workflow Trigger Event Contract

งานชิ้นนี้ ต้องเรียก workflow engine ตามตารางด้านล่าง · ชื่อ function ยึด API 8 ตัวของ `@srm/glb-workflow` ตามชีต `Detail` ของ TSM SRM LLDD SBP workflow ไฟล์1.2 — รายละเอียด signature และตารางที่ engine เขียน ดู LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3

จุดที่เรียก (call site)	Engine function	พารามิเตอร์หลัก	กติกา / transaction boundary
หลัง insert เอกสารสำเร็จ (Open first task)	`initializeWorkflow` แล้วต่อด้วย `addPreApprover`	versionId, userId, referenceId, stateId = `06`	อยู่ใน transaction boundary เดียวกับการสร้างเอกสาร — engine fail ต้อง rollback เอกสาร

- • กติกาเหล็ก: ตาราง `sps_store.workflow_*` (13 ตาราง) เป็นของ lib — SGI R เท่านั้น ห้าม INSERT/UPDATE/DELETE ตรงในทุกกรณี
- ทุกการเรียก engine ต้องผ่านตัวห่อกลาง `WorkflowGateway` ที่นิยามใน LLDD-BE-API-Common-Contracts.pdf (timeout · retry · map error เข้า envelope) ห้าม import lib ตรงจาก service
- unit test ต้อง mock engine และครอบอย่างน้อย: เรียกสำเร็จ · engine โยน error แล้ว rollback ฝั่ง SGI ครบ · เรียกซ้ำด้วย referenceId เดิมไม่เกิดผลซ้ำ

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Create document	POST	document.service.create	create doc + first workflow task
Update document section	PUT	document.service.updateSections	save editable sections

7. API Contract

POST /api/v1/sgi/document

Create document API

Request


```
{
  "source": "MANUAL",
  "impactMonth": "2026-07",
  "statementPeriod": "2026-07",
  "impactedStoreCode": "01234",
  "newStoreCode": "22864",
  "roundNo": 1,
  "reason": "สร้างเอกสารนอกเงื่อนไข",
  "requestId": "uuid"
}
```


Request Field Schema

Field	Type	Required	Constraint / Meaning
source	string	Yes	UTF-8; use value domain described by endpoint purpose
impactMonth	string	Yes	ISO-8601 ค.ศ.; nullable only when type includes null
statementPeriod	string	Yes	UTF-8; use value domain described by endpoint purpose
impactedStoreCode	string	Yes	exactly 5 digits; preserve leading zero
newStoreCode	string	Yes	exactly 5 digits; preserve leading zero
roundNo	integer	Yes	UTF-8; use value domain described by endpoint purpose
reason	string	Yes	trimmed UTF-8 Thai text; required by operation/business rule
requestId	string	Yes	UTF-8; use value domain described by endpoint purpose

Response


```
{
  "docNo": "2026/00001",
  "statusCode": "06",
  "message": "created"
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
docNo	string	Yes	ค.ศ. YYYY/xxxxx

Field	Type	Required	Constraint / Meaning
statusCode	string	Yes	canonical code; do not replace with display label
message	string	Yes	UTF-8; use value domain described by endpoint purpose

PUT /api/v1/sgi/document/{docNo}

Update document partial sections

Request


```
{
  "versionNo": 3,
  "newStores": [
    {
      "id": 9001,
      "newStoreCode": "00990",
      "compensatePercent": 60,
      "compensationAmount": 18000.0,
      "sourceSystem": "ALLMAP"
    },
    {
      "newStoreCode": "01180",
      "compensatePercent": 40,
      "compensationAmount": 12000.0,
      "sourceSystem": "USER",
      "id": 9001
    }
  ],
  "competitors": [
    {
      "id": 8801,
      "competitorCode": "01",
      "impactDate": "2026-07-15"
    },
    {
      "competitorCode": "07",
      "impactDate": "2026-07-20",
      "id": 8801
    }
  ],
  "externalFactors": [
    {
      "id": 4402,
      "factorCode": "F03",
      "dateFrom": "2026-07-01",
      "dateTo": "2026-07-31"
    },
    {
      "factorCode": "F09",
      "dateFrom": "2026-07-10",
      "dateTo": null,
      "id": 4402
    }
  ]
}
```


Request Field Schema

Field	Type	Required	Constraint / Meaning
versionNo	integer	Yes	เลขเวอร์ชันเอกสาร — ● ต้องส่งกลับใน PUT `/sgi/document/{docNo}` (optimistic lock · ไม่ตรงคืน 409 `STALE_VERSION`) · เพิ่ม 2026-09-22 · ของเดิม response ของ GET ไม่มีค่านี้ หน้าจอจึงสร้าง payload ที่ผ่าน lock ไม่ได้เลย = ปุ่มบันทึกใช้งานไม่ได้ตั้งแต่ต้น · ● response ของ PUT ต้องคืนเลขเวอร์ชันใหม่เป็นตัวเลข (วัดบน dev 2026-09-30 ของเดิมคืน `null` เพราะอ่านผล UPDATE … RETURNING ผิดรูป) ไม่งั้นการบันทึกครั้งถัดไปส่งเวอร์ชันค้างแล้วชน 409 ทุกครั้ง
newStores	array<object>	Yes	JSON array; element type shown in Type column
newStores[].id	integer	Yes	id ของแถว — ใช้เป็น key ของตารางบนหน้าจอ (เพิ่ม 2026-09-22)
newStores[].newStoreCode	string	Yes	exactly 5 digits; preserve leading zero
newStores[].compensatePercent	integer	Yes	number 0..100 with 2 decimals
newStores[].compensationAmount	number	Yes	number >= 0 with 2 decimals · `sgi_document_new_stores.compensation_amount` (คนละคอลัมน์กับ `compensationHistories[].compensationAmount` = `sgi_compensation_histories.compensate_amount`)
newStores[].sourceSystem	string	Yes	ALLMAP = ระบบดึงมาเอง · USER = ผู้ใช้คีย์เพิ่ม · ▲ ข้อมูลที่ย้ายมาจาก K2 มีค่า K2 ปนอยู่ด้วย (วัดจากฐาน dev 2026-09-22) หน้าจอจึงต้องถือว่า "ค่าที่ไม่ใช่ USER = ระบบดึงมา" ไม่ใช่เทียบเท่ากับ ALLMAP ตรงตัว
competitors	array<object>	Yes	JSON array; element type shown in Type column
competitors[].id	integer	No	id ของแถว — ● ต้องส่งกลับใน PUT เพื่อแยกแถวเดิม (UPDATE) ออกจากแถวที่ผู้ใช้เพิ่มใหม่ (INSERT) · BE sync แบบ แถวเดิมที่ไม่ถูกส่งมา = ถูกลบ (เพิ่ม 2026-09-22) · ของเดิม response ไม่มี id หน้าจอจึงรักษาตัวตนของแถวไม่ได้ — กดบันทึกครั้งเดียวจะกลายเป็นลบทั้งชุดแล้วสร้างใหม่
competitors[].competitorCode	string	Yes	รหัสแบรนด์คู่แข่งจาก master 01–12 เท่านั้น (ห้าม free text)
competitors[].impactDate	string	Yes	ISO-8601 ค.ศ.; nullable only when type includes null
externalFactors	array<object>	Yes	JSON array; element type shown in Type column
externalFactors[].id	integer	No	id ของแถว — เหตุผลเดียวกับ `competitors[].id` (เพิ่ม 2026-09-22)
externalFactors[].factorCode	string	Yes	รหัสปัจจัยภายนอกจาก master (sgi_external_factors.factor_code)

Field	Type	Required	Constraint / Meaning
externalFactors[].dateFrom	string	Yes	UTF-8; use value domain described by endpoint purpose
externalFactors[].dateTo	string	No	UTF-8; use value domain described by endpoint purpose

Response

```
{
  "message": "saved",
  "versionNo": 4
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
message	string	Yes	UTF-8; use value domain described by endpoint purpose
versionNo	integer	Yes	เลขเวอร์ชันเอกสาร — • ต้องส่งกลับใน PUT `/sgi/document/{docNo}` (optimistic lock · ไม่ตรงคืน 409 `STALE_VERSION`) · เพิ่ม 2026-09-22 · ของเดิม response ของ GET ไม่มีค่านี้ หน้าจอจึงสร้าง payload ที่ผ่าน lock ไม่ได้เลย = ปุ่มบันทึกใช้งานไม่ได้ตั้งแต่ต้น · • response ของ PUT ต้องคืนเลขเวอร์ชันใหม่เป็นตัวเลข (วัดบน dev 2026-09-30 ของเดิมคืน `null` เพราะอ่านผล UPDATE … RETURNING ผิดรูป) ไม่งั้นการบันทึกครั้งถัดไปส่งเวอร์ชันค้างแล้วชน 409 ทุกครั้ง

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช่งานสร้างหน้า Database, ไม่ใช่งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
sgi_compensation_documents	R/W	สร้างหัวเอกสารและแก้ไข section หลัก
workflow_transaction / workflow_approver (@srm/glb-workflow)	W (ผ่าน lib)	เปิด workflow งานแรกตอนสร้างเอกสารด้วย initializeWorkflow() + addPreApprover() — ห้าม INSERT ตรง
sgi_document_new_stores	R/W	ร้านเปิดใหม่และ % ชดเชย
sgi_document_competitors	R/W	ร้านคู่แข่งในเอกสาร
sgi_document_running_numbers	R/W	ตัวนับเลขเอกสารต่อปี ค.ศ. — ออกเลข YYYY/xxxxx แบบ atomic (INSERT … ON CONFLICT DO UPDATE … RETURNING)
sgi_fgi_impact_processes	R	หารอบของเอกสารเพื่อคำนวณรอบ/ครั้งที่ตามสูตรเดิม (`roundLoopBasis`) และกันสร้างซ้ำ (เพิ่ม 2026-10-27 — SQL แตะอยู่แล้วแต่ §8 ไม่ได้ประกาศ)

Table / Object	R/W	Usage
sgi_fgi_impact_compensations	R	หารอบชดเชยตั้งต้นของเอกสารที่จะสร้าง — ค้นด้วย (impacted_store_code, compensate_month, compensate_seq) แล้วได้ `impact_process_id` + `impact_compensation_id` ซึ่งเป็น NOT NULL ทั้งคู่ บนหัวเอกสาร · คีย์อ้างอิงจาก Job 8 ที่ map `compensate_seq` = roundNo · `compensate_seq_no` = loopNo · เจอมากกว่า 1 แถวต้องไม่เดา (`impact_compensation_id` เป็น UNIQUE บนเอกสาร) ให้ตอบ CONFLICT ให้ผู้เรียกระบุมาเอง
sgi_document_cost_details	R/W	ยอดชดเชยแยกรายเดือน/รายร้านเปิดใหม่ (cost_year/cost_month · cost_target · cost_amount · _n / _nc)
sgi_document_external_factors	R/W	ปัจจัยภายนอกในเอกสาร
sgi_compensation_documents unique guard	R	กัน duplicate ด้วย business key: impact_process_id หรือ source + impacted_store_code + impact_month + new_store_code + round_no
sgi_competitors	R	ตรวจว่ารหัสแบรนด์คู่แข่งที่ผู้ใช้เลือกมีอยู่ใน master จริงก่อน INSERT ลง `sgi_document_competitors` — อ่านสดจากฐาน ห้าม hardcode ช่วงรหัส เพราะ `POST /sgi/master/competitors` เพิ่มแบรนด์ใหม่ได้ตลอด

9. Skeleton Code (store-backend + BFF)

โครงโค้ดตั้งต้นของเอกสารฉบับนี้ ยึด convention จริงของ `srm-sps-spsap-store-backend` (NestJS 11 + TypeORM, schema `sps_store`, custom provider `DATA_SOURCE` ที่ route SELECT ไป slave pool) และ `srm-sps-spsap-sbp-bff` (ไม่มี DB, forward ผ่าน client service). ทุกจุดที่ต้องเติมกำกับด้วย `// TODO:` และ response ทุกเส้นถูกห่อเป็น `{success, data}` โดย ResponseInterceptor อยู่แล้ว จึงห้าม service ห่อซ้ำ

9.1 ผังไฟล์ที่ต้องสร้าง

Path	หน้าที่
store-backend · src/modules/sgi-document-create-update/sgi-document-create-update.controller.ts	route ทั้งหมดของเอกสารนี้ (2 เส้น) + `@UseGuards(HttpHeaderGuard)` + `@UserId()`
store-backend · src/modules/sgi-document-create-update/sgi-document-create-update.service.ts	business logic — inject `'DATA_SOURCE'` แล้วยิง raw SQL, mutation ใช้ QueryRunner transaction
store-backend · src/modules/sgi-document-create-update/sgi-document-create-update.sql.ts	เก็บ SQL ต่อ endpoint (คัดจากหัวข้อ 10) แยกออกจาก service ให้ทดสอบ/รีวิวง่าย · คีย์ = ชื่อ handler เช่น `getSgiMasterFactors` · บล็อกที่มีหลาย statement ให้แยกเป็นหลายคีย์ โดยเติมท้ายชื่อให้สื่อความ เช่น DELETE master ที่มี 2 statement → `removeSgiMasterFactorsByCodeInUse` (SELECT ตรวจการใช้งาน) + `removeSgiMasterFactorsByCode` (DELETE)
store-backend · src/modules/sgi-document-create-update/dto/sgi-document-create-update.dto.ts	DTO + class-validator ตาม validation ในหัวข้อฟิลด์ของเอกสารนี้
store-backend · src/modules/sgi-document-create-update/sgi-document-create-update.module.ts	ประกอบ controller/service/providers แล้ว register ที่ `app.module.ts`
store-backend · src/entitys/sgi-compensation-documents.entity.ts	entity ของ `sgi_compensation_documents` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation) — entity ร่วมหลายเอกสาร: ประกาศครั้งเดียวแล้วอ้างอิง อย่าสร้างซ้ำ
store-backend · src/entitys/sgi-document-new-stores.entity.ts	entity ของ `sgi_document_new_stores` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation)
store-backend · src/entitys/sgi-document-competitors.entity.ts	entity ของ `sgi_document_competitors` (`@Entity({schema: process.env.DB_SCHEMA})`, ไม่ประกาศ relation)

Path	หน้าที่
store-backend · src/providers/sgi/sgi.ts	repository provider แบบ factory ผูก token string กับ DATA_SOURCE — ไฟล์รวมของทุกเอกสาร BE ให้ merge array เพิ่ม ห้ามเขียนทับ
store-backend · sql/deploy-sgi-document-create-update.sql	DDL production แบบ idempotent (ทีมนี้ไม่ใช้ migration เป็นหลัก)
BFF · src/common/client-services/sgi-client.service.ts	client ต่อจาก `BaseClientService` ตั้ง baseUrl + `x-api-key` ตอน `onModuleInit`
BFF · src/modules/sgi-document-create-update/sgi-document-create-update.controller.ts	route ฝั่ง BFF prefix /bff/sgi/… + `@UseGuards(AuthGuard('jwt'))`
BFF · src/modules/sgi-document-create-update/sgi-document-create-update.service.ts	แนบ `x-user-id` / `x-user-group-id` / `x-user-permissions` แล้ว forward ไป backend

9.2 Controller (store-backend)

```
// src/modules/sgi-document-create-update/sgi-document-create-update.controller.ts
import { Body, Controller, Param, Post, Put, UseGuards } from '@nestjs/common';
import { HttpHeaderGuard } from '../../guards/http-header.guard';
import { UserId } from '../../common/decorators/user-id.decorator';
import { SgiDocumentCreateUpdateService } from './sgi-document-create-update.service';
import { CreateSgiDocumentBodyDto, UpdateSgiDocumentByDocNoBodyDto } from './dto/sgi-document-create-update.dto';

// LLDD BE - API Document Create and Update
// BFF เรียกด้วย x-api-key และแนบ x-user-id / x-user-group-id / x-user-permissions มาให้
@Controller('document')
@UseGuards(HttpHeaderGuard)
export class SgiDocumentCreateUpdateController {
  constructor(private readonly service: SgiDocumentCreateUpdateService) {}

  // POST /api/v1/sgi/document — Create document API
  @Post()
  createSgiDocument(@Body() body: CreateSgiDocumentBodyDto, @UserId() userId: string) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.createSgiDocument(body, userId);
  }

  // PUT /api/v1/sgi/document/{docNo} — Update document partial sections
  @Put(':docNo')
  updateSgiDocumentByDocNo(
    @Param('docNo') docNo: string,
    @Body() body: UpdateSgiDocumentByDocNoBodyDto,
    @UserId() userId: string,
  ) {
    // TODO: ตรวจ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.updateSgiDocumentByDocNo(docNo, body, userId);
  }
}
```

9.3 DTO + Validation

```
// src/modules/sgi-document-create-update/dto/sgi-document-create-update.dto.ts
import { Type } from 'class-transformer';
import {
  IsArray, IsBoolean, IsIn, IsInt, IsNotEmpty, IsNumber, IsObject, IsOptional,
  IsString, Matches, Max, MaxLength, Min, ValidateNested,
} from 'class-validator';

// ValidationPipe ระดับ global ตั้ง whitelist + forbidNonWhitelisted + transform ไว้แล้ว (main.ts)
// property ที่ไม่ประกาศที่นี่จะถูก reject เป็น 400 อัตโนมัติ

// body ของ POST /api/v1/sgi/document
export class CreateSgiDocumentBodyDto {
  /** แยกแหล่งสร้างเอกสาร */
  @IsNotEmpty()
  @IsString()
  @IsIn(['MANUAL', 'FS'])
  source: string;

  @IsNotEmpty()
  @IsString()
  impactMonth: string;

  @IsNotEmpty()
  @IsString()
  statementPeriod: string;

  @IsNotEmpty()
```

```
  @IsString()
  impactedStoreCode: string;

  @IsNotEmpty()
  @IsString()
  newStoreCode: string;

  @IsNotEmpty()
  @Type(() => Number)
  @IsInt()
  roundNo: number;

  // TODO: เพิ่ม property ที่เหลือของ payload นี้ให้ครบตามหัวข้อฟิลด์ของเอกสารนี้
}

// 1 แถวของ `newStores` — property ที่ไม่ได้มีครบทุกแถวถือเป็น optional
export class NewStoreItemDto {
  @IsNotEmpty()
  @Type(() => Number)
  @IsInt()
  id: number;

  @IsNotEmpty()
  @IsString()
  newStoreCode: string;

  @IsNotEmpty()
  @Type(() => Number)
  @IsInt()
  compensatePercent: number;

  @IsNotEmpty()
  @Type(() => Number)
  @IsInt()
  compensationAmount: number;

  /** **B5** ที่มาของแถวร้านเปิดใหม่ — `ALLMAP` ระบบ default ให้อัตโนมัติ (Job 9) · `USER` เจ้า… */
  @IsNotEmpty()
  @IsString()
  sourceSystem: string;
}

// 1 แถวของ `competitors` — property ที่ไม่ได้มีครบทุกแถวถือเป็น optional
export class CompetitorItemDto {
  @IsNotEmpty()
  @Type(() => Number)
  @IsInt()
  id: number;

  @IsNotEmpty()
  @IsString()
  competitorCode: string;

  @IsNotEmpty()
  @IsString()
  impactDate: string;
}

// 1 แถวของ `externalFactors` — property ที่ไม่ได้มีครบทุกแถวถือเป็น optional
export class ExternalFactorItemDto {
  @IsNotEmpty()
  @Type(() => Number)
  @IsInt()
  id: number;

  @IsNotEmpty()
  @IsString()
  factorCode: string;

  @IsNotEmpty()
  @IsString()
  dateFrom: string;

  /** ส่ง null ได้ */
  @IsOptional()
  @IsString()
  dateTo?: string;
}

// body ของ PUT /api/v1/sgi/document/{docNo}
export class UpdateSgiDocumentByDocNoBodyDto {
  @IsNotEmpty()
  @Type(() => Number)
```

```
  @IsInt()
  versionNo: number;

  @IsNotEmpty()
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => NewStoreItemDto)
  newStores: NewStoreItemDto[];

  @IsNotEmpty()
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => CompetitorItemDto)
  competitors: CompetitorItemDto[];

  @IsNotEmpty()
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => ExternalFactorItemDto)
  externalFactors: ExternalFactorItemDto[];
}
```

9.4 Service (inject `DATA_SOURCE` + raw SQL)

service ประกาศ method ครบทุกเส้นที่ controller เรียก และ signature มาจากแหล่งเดียวกับ controller (จำนวน/ลำดับพารามิเตอร์จึงตรงกันเสมอ) — เส้นที่ยังไม่ได้ implement เป็น stub ที่ `throw new NotImplementedException(...)` ให้ TypeScript compile ผ่านตั้งแต่วันแรก

```
// src/modules/sgi-document-create-update/sgi-document-create-update.service.ts
import { Inject, Injectable, Logger, NotImplementedException } from '@nestjs/common';
import { DataSource } from 'typeorm';
import { SgiException } from '../sgi/exceptions/sgi.exception';
import { WorkflowService } from '../workflow/workflow.service';
import { SGI_SQL } from './sgi-document-create-update.sql';

@Injectable()
export class SgiDocumentCreateUpdateService {
  private readonly logger = new Logger(SgiDocumentCreateUpdateService.name);
  // versionId ของ workflow ประกันรายได้ (ตั้งใน env เหมือน COOPERATION_WORKFLOW_VERSION_ID)
  private readonly versionId = Number(process.env.SGI_WORKFLOW_VERSION_ID);

  constructor(
    // DATA_SOURCE override query(): SELECT/WITH ไป slave pool, write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly workflow: WorkflowService,
  ) {}

  // POST /api/v1/sgi/document — Create document API
  // mutation ต้องอยู่ใน transaction เดียว (ไม่มี audit ของ master แล้ว · 2026-08-07)
  async createSgiDocument(body: CreateSgiDocumentBodyDto, userId: string) {
    const runner = this.dataSource.createQueryRunner();
    await runner.connect();
    await runner.startTransaction();
    try {
      // TODO: lock แถวเป้าหมายของ sgi_compensation_documents ด้วย SELECT ... FOR UPDATE ก่อนเขียน
      const [current] = await runner.query(SGI_SQL.createSgiDocumentLock, [body.impactedStoreCode]);
      if (!current) {
        throw new SgiException('NOT_FOUND', 'ไม่พบข้อมูลที่ต้องการ');
      }
      await runner.query(SGI_SQL.createSgiDocument, [/* TODO: ผูกค่าจาก body */]);
      await runner.commitTransaction();
      // ▲ workflow engine อยู่คนละ DataSource ('workflow-connection' ของ @srm/glb-workflow)
      // จึง **atomic รวมกับ transaction ข้างบนไม่ได้** — ต้อง commit ฝั่ง SGI ให้เสร็จก่อน
      // แล้วค่อย eventWorkflow (idempotency key = referenceId = docNo)
      // TODO: เรียก workflow use case ตามตารางหัวข้อ Workflow ด้านล่าง + retry
      // TODO: ถ้า eventWorkflow ล้มเหลว ต้องมี compensating action และบันทึกผลลง
      // sgi_consideration_logs เพื่อให้ job reconcile ตามเก็บได้
      return { message: 'saved' };
    } catch (error) {
      await runner.rollbackTransaction();
      this.logger.error(error);
      throw error;
    } finally {
      await runner.release();
    }
  }

  // PUT /api/v1/sgi/document/{docNo} — Update document partial sections
  async updateSgiDocumentByDocNo(docNo: string, body: UpdateSgiDocumentByDocNoBodyDto, userId: string) {
    // TODO: implement ตาม business rule ของ PUT /api/v1/sgi/document/{docNo}
    // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'PUT /api/v1/sgi/document/{docNo}')
```

```
    throw new NotImplementedException('updateSgiDocumentByDocNo ยังไม่ implement');
  }
}
```

9.5 Workflow (`@srm/glb-workflow`)

✓ ชื่อ function ของ engine — ยึด LLDD ของ lib (ยืนยันแล้ว 2026-08-14) · API จริงคือ 8 ตัวตามชีต `Detail` ของ `SBP/TSM-SRM-LLDD SBP workflow 1.2.xlsx` (เอกสารของ lib เอง): `initializeWorkflow` · `eventWorkflow` · `getPermissionEvents` · `getHistory` · `getTransaction` · `getPendingFlowByUser` · `getWorkflowsByUser` · `addPreApprover` · ชื่อที่เคยขัดกันไม่ใช่ชื่อ API — *Trigger Event* เป็นชื่อหัวข้อขั้นตอนภายใน `eventWorkflow` และ `*UseCase` เป็น class ที่ store-backend ห่อไว้ใช้เอง (ดู `LLDD-BE-Workflow-Engine-Definition.pdf` หัวข้อ 5.3)

Endpoint	Use case ที่ต้องเรียก	เหตุผล
(อ่านสถานะประกอบ)	getTransaction()	อ่านสถานะปัจจุบันของเอกสารเพื่อประกอบ response

```
// src/modules/sgi-document-create-update/sgi-document-create-update.workflow.ts (หรือรวมไว้ใน service เดียวกัน)
// WorkflowService = wrapper ของ @srm/glb-workflow ที่ store-backend มีอยู่แล้ว
// (DataSource แยกชื่อ 'workflow-connection', ทุก use case ห่อด้วย TypeOrmUnitOfWork)

 // สถานะปัจจุบันของเอกสาร
 const trx = await this.workflow.getTransaction({ versionId: this.versionId, referenceId: docNo });
 // TODO: map currentState -> statusCode/statusName ที่ FE ใช้
```

9.6 Entity (TypeORM)

```
// src/entitys/sgi-compensation-documents.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_compensation_documents', schema: process.env.DB_SCHEMA })
export class CompensationDocument {
 @PrimaryColumn({ name: 'id', type: 'bigint' })
 id: number;

 @Column({ name: 'doc_no', type: 'varchar', length: 10, nullable: true })
 docNo?: string;

 @Column({ name: 'year', type: 'int', nullable: true })
 year?: number;

 @Column({ name: 'running_no', type: 'int', nullable: true })
 runningNo?: number;

 @Column({ name: 'impact_process_id', type: 'bigint' })
 impactProcessId: number;

 @Column({ name: 'impact_compensation_id', type: 'bigint' })
 impactCompensationId: number;

 @Column({ name: 'impacted_store_code', type: 'varchar', length: 5 })
 impactedStoreCode: string;

 @Column({ name: 'impact_month', type: 'char', length: 7, nullable: true })
 impactMonth?: string;

 @Column({ name: 'new_store_code', type: 'varchar', length: 5, nullable: true })
 newStoreCode?: string;

 @Column({ name: 'round_no', type: 'int', nullable: true })
 roundNo?: number;

 @Column({ name: 'loop_no', type: 'int', nullable: true })
 loopNo?: number;

 @Column({ name: 'source', type: 'varchar', length: 20, default: 'FS' })
 source: string;

 @Column({ name: 'status_code', type: 'varchar', length: 2, default: '06' })
 statusCode: string;

 @Column({ name: 'current_section_code', type: 'varchar', length: 2, nullable: true })
 currentSectionCode?: string;

 @Column({ name: 'total_compensation_amount', type: 'numeric', precision: 14, scale: 2, default: 0 })
 totalCompensationAmount: string;

 @Column({ name: 'allmap_url', type: 'varchar', length: 500, nullable: true })
 allmapUrl?: string;
```

```
  @Column({ name: 'statement_id', type: 'varchar', length: 500, nullable: true })
  statementId?: string;

  @Column({ name: 'statement_date', type: 'date', nullable: true })
  statementDate?: Date;

  @Column({ name: 'account_year', type: 'int', nullable: true })
  accountYear?: number;

  @Column({ name: 'account_month', type: 'int', nullable: true })
  accountMonth?: number;

  @Column({ name: 'approver_snapshot', type: 'jsonb', nullable: true })
  approverSnapshot?: Record<string, unknown>;

  @Column({ name: 'draft_result', type: 'varchar', length: 100, nullable: true })
  draftResult?: string;

  @Column({ name: 'draft_comment', type: 'text', nullable: true })
  draftComment?: string;

  @Column({ name: 'draft_section_code', type: 'varchar', length: 2, nullable: true })
  draftSectionCode?: string;

  @Column({ name: 'draft_by', type: 'varchar', length: 30, nullable: true })
  draftBy?: string;

  @Column({ name: 'draft_at', type: 'timestamp', nullable: true })
  draftAt?: Date;

  @Column({ name: 'version_no', type: 'int', default: 1 })
  versionNo: number;

  @Column({ name: 'created_by', type: 'varchar', length: 30 })
  createdBy: string;

  @Column({ name: 'created_at', type: 'timestamp' })
  createdAt: Date;

  @Column({ name: 'updated_by', type: 'varchar', length: 30, nullable: true })
  updatedBy?: string;

  @Column({ name: 'updated_at', type: 'timestamp', nullable: true })
  updatedAt?: Date;

  // entity ชุดนี้ generate จาก DDL ใน LLDD-Database.pdf §5.2–5.4 โดยตรง — คอลัมน์/ชนิด/nullable ตรงกันเสมอ
  // ไม่ประกาศ relation ตาม convention ของทีม (join ด้วย raw SQL)
}

// src/entitys/sgi-document-new-stores.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_document_new_stores', schema: process.env.DB_SCHEMA })
export class DocumentNewStore {
  @PrimaryColumn({ name: 'id', type: 'bigint' })
  id: number;

  @Column({ name: 'doc_no', type: 'varchar', length: 10 })
  docNo: string;

  @Column({ name: 'new_store_code', type: 'varchar', length: 5 })
  newStoreCode: string;

  @Column({ name: 'distance_km', type: 'numeric', precision: 8, scale: 3, nullable: true })
  distanceKm?: string;

  @Column({ name: 'compensate_percent', type: 'numeric', precision: 7, scale: 4 })
  compensatePercent: string;

  @Column({ name: 'compensation_amount', type: 'numeric', precision: 14, scale: 2, default: 0 })
  compensationAmount: string;

  @Column({ name: 'source_system', type: 'varchar', length: 30 })
  sourceSystem: string;

  @Column({ name: 'source_row_id', type: 'bigint', nullable: true })
  sourceRowId?: number;

  @Column({ name: 'updated_at', type: 'timestamp' })
  updatedAt: Date;

  // entity ชุดนี้ generate จาก DDL ใน LLDD-Database.pdf §5.2–5.4 โดยตรง — คอลัมน์/ชนิด/nullable ตรงกันเสมอ
```

```
  // ไม่ประกาศ relation ตาม convention ของทีม (join ด้วย raw SQL)
}
```

ตารางที่เหลือของเอกสารนี้ (`sgi_document_competitors`, `sgi_document_running_numbers`, `sgi_fgi_impact_processes`, `sgi_fgi_impact_compensations`, `sgi_document_cost_details`, `sgi_document_external_factors`, `sgi_competitors`) ใช้รูปแบบ entity เดียวกัน — คอลัมน์อ้างจาก `Database design`

ตารางที่ ไม่ต้องสร้าง entity เพราะใช้ของระบบเดิม/workflow engine:

Object	R/W	ใช้ของระบบเดิมตัวไหน
workflow_transaction	W (ผ่าน lib)	workflow engine @srm/glb-workflow
workflow_approver	W (ผ่าน lib)	workflow engine @srm/glb-workflow

9.7 Repository Providers + Module wiring

```
// src/providers/sgi/sgi.ts — repository provider แบบ factory (ไม่ใช้ TypeOrmModule.forFeature)
// convention ของโฟลเดอร์ providers คือ 1 ไฟล์ต่อโดเมน ตั้งชื่อตามโดเมน (business_user/business_user.ts,
// common_code/common_code.ts …) ไม่ใช่ index.ts
//
// ▲ ไฟล์นี้ใช้ร่วมกันทุกเอกสาร BE ของ SGI — ให้ **merge array เพิ่ม** เข้าไฟล์เดิม ห้ามเขียนทับ
// (ชื่อ const แยกต่อเอกสารไว้แล้วเพื่อไม่ให้ชนกัน)
import { DataSource } from 'typeorm';
import { CompensationDocument } from '../../entitys/sgi-compensation-documents.entity';
import { DocumentNewStore } from '../../entitys/sgi-document-new-stores.entity';
import { DocumentCompetitor } from '../../entitys/sgi-document-competitors.entity';

export const sgiDocumentCreateUpdateProviders = [
  {
    provide: 'SGI_COMPENSATION_DOCUMENT_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(CompensationDocument),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'SGI_DOCUMENT_NEW_STORE_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(DocumentNewStore),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'SGI_DOCUMENT_COMPETITOR_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(DocumentCompetitor),
    inject: ['DATA_SOURCE'],
  },
];

// src/modules/sgi-document-create-update/sgi-document-create-update.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { DatabaseModule } from '../../database/database.module';
// UserContextMiddleware อ่าน header x-user-id แล้วเซ็ต request.userId ที่ @UserId() ใช้
// — app.module.ts **ไม่ได้** apply แบบ global (มีแค่ HttpContext/LoggerContext) แต่ละโมดูลต้อง apply เอง
// (ดู evaluation-process.module.ts / inform-evaluate.module.ts / cooperation-request.module.ts)
import { UserContextMiddleware } from '../../common/middleware/user-context.middleware';
import { WorkflowModule } from '../workflow/workflow.module';
import { sgiDocumentCreateUpdateProviders } from '../../providers/sgi/sgi';
import { SgiDocumentCreateUpdateController } from './sgi-document-create-update.controller';
import { SgiDocumentCreateUpdateService } from './sgi-document-create-update.service';

@Module({
  imports: [DatabaseModule, WorkflowModule],
  controllers: [SgiDocumentCreateUpdateController],
  providers: [SgiDocumentCreateUpdateService, ...sgiDocumentCreateUpdateProviders],
  exports: [SgiDocumentCreateUpdateService],
})
export class SgiDocumentCreateUpdateModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    // ถ้าไม่ apply ตรงนี้ userId จะเป็น undefined เงียบ ๆ ทุก endpoint
    consumer.apply(UserContextMiddleware).forRoutes(SgiDocumentCreateUpdateController);
  }
}
// TODO: register module นี้ใน app.module.ts (imports) พร้อมกับโมดูล SGI ตัวอื่น
```

9.8 BFF Proxy (module + controller + client service)

BFF ยังไม่มีฟีเจอร์ประกันรายได้เลย จึงต้องสร้าง module ใหม่ + client service ใหม่ทั้งชุด และเลือก prefix แบบเดียวทั้งโมดูล (ที่นี่ใช้ /bff/sgi/…) เพื่อไม่ให้ปนแบบที่มี/ไม่มี `/bff` เหมือนโมดูลเดิม

```
// BFF — ไม่ต้องมี client ของ SGI แยก
//
// ● แก้ 2026-09-22 — skeleton เดิมเสนอ `SgiClientService` เป็น class แยกที่ *ชี้ไป store backend*
// ของจริงในรีโปกลับชี้ `config.api.sgi.url` (ค่าตั้งต้น http://localhost:3010)
// ซึ่ง **ไม่มี backend นั้นอยู่จริง** และ `API_SGI_BACKEND_URL` ไม่ได้ตั้งไว้ที่ไหนเลย
// → 12 เส้น (รวม 5 เส้นที่ FE เรียกจริง) ยิงไม่ถึงปลายทาง (พบตอนรันจริง)
// มติ DP-10 ระบุว่า SGI **ไม่แยก backend** — อยู่ใน srm-sps-spsap-store-backend
// จึงใช้ `StoreClientService` ที่มีอยู่แล้วตรง ๆ ไม่สราง client ตัวที่สอง
// (`tools/check_docs.py` มีกฎห้ามโมดูล SGI ของ BFF ใช้ SgiClientService)
//
// header: ใช้ `OperatingManagementService.header(req)` ชุดกลางเท่านั้น ห้ามประกอบเอง
// ตัวประกอบที่เขียนเองในโมดูลส่ง header ไม่ครบ (ขาด x-user-full-name / accept-language)
// และ serialize x-user-permissions คนละรูปแบบกับฝั่งรับที่ใช้จริง (JSON.parse)
//
// path ที่ forward ให้ใช้ `/api/v1/sgi/...` เสมอ — controller ฝั่ง BE บางตัวประกาศ base path
// เป็น array 2 ค่า แต่บางตัวมีเฉพาะ canonical
// BaseClientService และ { success, data } ให้แล้ว — service ฝั่ง BFF จึงได้ data ตรง ๆ
// src/modules/sgi-document-create-update/sgi-document-create-update.service.ts (BFF)
import { Injectable } from '@nestjs/common';
import { StoreClientService } from '@common/client-services/store-client.service';
import { OperatingManagementService } from '@modules/operating-management/operating-management.service';

@Injectable()
export class SgiDocumentCreateUpdateBffService {
  constructor(
    private readonly client: StoreClientService,
    private readonly operatingManagementService: OperatingManagementService,
  ) {}

  // BFF ไม่มี DB — หน้าที่เดียวคือแนบ user context แล้ว forward
  // ▲ ต้อง unwrap envelope ของ store-backend 1 ชั้นก่อนคืน (ยืนยันจากโค้ดจริง 2026-09-04):
  // ResponseInterceptor ระดับ global ของ BFF ห่อผลลัพธ์เป็น { success, data, requestId } อีกที
  // ถ้าคืน { success, data } ดิบมา FE จะได้ data.data.data — BaseClientService จึงต้องคืน .data.data
  //
  // ● แก้ 2026-09-22 — เดิม skeleton ให้ประกอบ header เองด้วย `userHeaders(user)` 3 ตัว
  // ส่งไม่ครบตาม LLDD-BE-Integration-SBP-Platform.pdf §5.1 (ขาด x-user-full-name · accept-language)
  // และ serialize x-user-permissions เป็น comma-separated ขณะที่ฝั่งรับที่ใช้จริง
  // (OperatingManagementHeaderGuard.parsePermissions) ทำ JSON.parse — คนละรูปแบบกัน
  // ให้ใช้ตัวประกอบกลางตัวเดียวทั้งระบบ และรับ `req` ไม่ใช่ `req.user`
  async createSgiDocument(body: any, req: any) {
    return this.client.post('/api/v1/sgi/document', body, { headers: await this.operatingManagementService.header(req) });
  }

  async updateSgiDocumentByDocNo(docNo: string, body: any, req: any) {
    return this.client.put(`/api/v1/sgi/document/${docNo}`, body, { headers: await this.operatingManagementService.header(req) });
  }
}

// ---------- src/modules/sgi-document-create-update/sgi-document-create-update.controller.ts (BFF) ----------
import { Body, Controller, Delete, Get, Param, Post, Put, Query, Req, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

// path เดียวกับที่ FE เรียก (apiClient baseURL รวม /api/v1 แล้ว) — ห้ามตั้งตามชื่อเอกสาร LLDD
@Controller('sgi/document')
@UseGuards(AuthGuard('jwt'))
export class SgiDocumentCreateUpdateBffController {
  constructor(private readonly service: SgiDocumentCreateUpdateBffService) {}

  // proxy ของ POST /api/v1/sgi/document
  @Post()
  createSgiDocument(@Body() body: any, @Req() req: any) {
    return this.service.createSgiDocument(body, req);
  }

  // proxy ของ PUT /api/v1/sgi/document/{docNo}
  @Put(':docNo')
  updateSgiDocumentByDocNo(@Param('docNo') docNo: string, @Body() body: any, @Req() req: any) {
    return this.service.updateSgiDocumentByDocNo(docNo, body, req);
  }
}
// TODO: register module ใน app.module.ts ของ BFF · imports ต้องมี ClientServiceModule + OperatingManagementModule
```

10. Database SQL

10.1 ตารางที่อ่าน/เขียน

Table / Object	R/W	Usage
sgi_compensation_documents	R/W	สร้างหัวเอกสารและแก้ไข section หลัก
sgi_document_new_stores	R/W	ร้านเปิดใหม่และ % ชดเชย
sgi_document_competitors	R/W	ร้านคู่แข่งในเอกสาร
sgi_document_running_numbers	R/W	ตัวนับเลขเอกสารต่อปี ค.ศ. — ออกเลข YYYY/xxxxx แบบ atomic (INSERT … ON CONFLICT DO UPDATE … RETURNING)
sgi_fgi_impact_processes	R	หารอบของเอกสารเพื่อคำนวณรอบ/ครั้งที่ตามสูตรเดิม (`roundLoopBasis`) และกันสร้างซ้ำ (เพิ่ม 2026-10-27 — SQL แตะอยู่แล้วแต่ §8 ไม่ได้ประกาศ)
sgi_fgi_impact_compensations	R	หารอบชดเชยตั้งต้นของเอกสารที่จะสร้าง — ค้นด้วย (impacted_store_code, compensate_month, compensate_seq) แล้วได้ `impact_process_id` + `impact_compensation_id` ซึ่งเป็น NOT NULL ทั้งคู่ บนหัวเอกสาร · คียอ้างอิงจาก Job 8 ที่ map `compensate_seq` = roundNo · `compensate_seq_no` = loopNo · เจอมากกว่า 1 แถวต้องไม่เดา (`impact_compensation_id` เป็น UNIQUE บนเอกสาร) ให้ตอบ CONFLICT ให้ผู้เรียกระบุมาเอง
sgi_document_cost_details	R/W	ยอดชดเชยแยกรายเดือน/รายร้านเปิดใหม่ (cost_year/cost_month · cost_target · cost_amount · _n / _nc)
sgi_document_external_factors	R/W	ปัจจัยภายนอกในเอกสาร
sgi_competitors	R	ตรวจว่ารหัสแบรนด์คู่แข่งที่ผู้ใช้เลือกมีอยู่ใน master จริงก่อน INSERT ลง `sgi_document_competitors` — อ่านสดจากฐาน ห้าม hardcode ช่วงรหัส เพราะ `POST /sgi/master/competitors` เพิ่มแบรนด์ใหม่ได้ตลอด
workflow_transaction	W (ผ่าน lib)	ใช้ของระบบเดิม: workflow engine @srm/glb-workflow
workflow_approver	W (ผ่าน lib)	ใช้ของระบบเดิม: workflow engine @srm/glb-workflow

10.2 SQL จริงต่อ Endpoint

POST /api/v1/sgi/document — Create document API

```
-- bind ตามลำดับ: $1=source · $2=storeCode · $3=month · $4=newStoreCode · $5=roundNo · $6=terminalStatuses · $7=docNo · $8=year · $9=runningNo · $10=statusInit · $11=section06 · $12=empId · $13=impactProcessId · $14=impactCompensationId
-- กันซ้ำเฉพาะเอกสาร active (SDD GI): เอกสารเดิมที่จบด้วยหยุดชดเชย/เห็นควรไม่ชดเชย เปิดเรื่องใหม่ได้
-- ● business key ที่ใช้จริงคือ 5 ช่องนี้ (ไม่ใช่ impact_process_id) เพราะผู้เรียกยังไม่มี id นั้นในมือ
SELECT 1 FROM sgi_compensation_documents
WHERE source = $1 /* source */ AND impacted_store_code = $2 /* storeCode */ AND impact_month = $3 /* month */
  AND new_store_code = $4 /* newStoreCode */ AND round_no = $5 /* roundNo */
  AND NOT (status_code = ANY($6 /* terminalStatuses */));

-- ● หา "รอบชดเชยตั้งต้น" ก่อน INSERT — impact_process_id + impact_compensation_id เป็น NOT NULL ทั้งคู่
-- (เพิ่ม 2026-09-23 · ของเดิมไม่เคยส่งสองค่านี้ → 23502 ทุกคำขอ ไม่เคยสร้างเอกสารได้เลย)
-- คียอ้างอิงจาก Job 8: compensate_seq = roundNo · compensate_seq_no = loopNo
SELECT c.id AS impact_compensation_id, c.impact_process_id, c.compensate_seq_no
FROM sgi_fgi_impact_compensations c
WHERE c.impacted_store_code = $2 /* storeCode */ AND c.compensate_month = $3 /* month */ AND c.compensate_seq = $5 /* roundNo */
ORDER BY c.compensate_seq_no;
-- ไม่เจอแถว → 400 VALIDATION พร้อมระบุร้าน/งวด/ครั้งที่
-- เจอ > 1 แถว → 409 CONFLICT **ห้ามเดา** — impact_compensation_id เป็น UNIQUE บนเอกสาร เลือกผิดคือผูกกับรอบผิด
```

```
-- ออกเลขที่ YYYY/xxxxx (running ต่อปี) แล้วสร้างเอกสาร + เปิด workflow งานแรก (Section 06)
INSERT INTO sgi_compensation_documents (doc_no, year, running_no, impacted_store_code, impact_month, new_store_code,
                                        round_no, source, status_code, current_section_code, created_by,
                                        impact_process_id, impact_compensation_id)
VALUES ($7 /* docNo */, $8 /* year */, $9 /* runningNo */, $2 /* storeCode */, $3 /* month */, $4 /* newStoreCode */,
       $5 /* roundNo */, $1 /* source */, $10 /* statusInit */, $11 /* section06 */, $12 /* empId */,
       $13 /* impactProcessId */, $14 /* impactCompensationId */);
-- ▲ duplicateGuard รัน **นอก transaction** (TOCTOU) — backstop จริงคือ UNIQUE ของ impact_compensation_id
-- ชนแล้วได้ 23505 ซึ่งต้องแปลงเป็น DUPLICATE_DOCUMENT ไม่ใช่ "กรุณาลองอีกครั้ง"
-- ▲ ไม่ INSERT ตาราง workflow เอง — เรียก @srm/glb-workflow (schema sps_store) ให้ library เขียนให้
-- initialize(versionId=:sgiVersionId, referenceId=:referenceId, userId=:empId)
-- addPreApprover(versionId, referenceId, stateId=:section06, approver, seq=1)
-- library เขียน sps_store.workflow_transaction / workflow_approver / workflow_history ให้เอง
-- referenceId = sgi_compensation_documents.id (surrogate · DP-1 ปิดแล้ว 2026-08-17)
-- ▲ sps_store.workflow_transaction ไม่มี PK/index → กันซ้ำต้องทำที่ application (DP-2)
-- ดู SBPGI vs existing system หัวข้อ 4
```

PUT /api/v1/sgi/document/{docNo} — Update document partial sections

```
-- bind ตามลำดับ: $1=now · $2=empId · $3=docNo · $4=versionNo · $5=pct · $6=amount · $7=newStoreCode · $8=date · $9=competitorId ·
$10=from · $11=to · $12=factorId · $13=competitorStoreCode · $14=brandCode · $15=impactDate · $16=factorCode · $17=dateFrom ·
$18=dateTo · $19=keepCompetitorIds · $20=keepFactorIds
-- ตรวจสิทธิ์ตาม role + current_section ก่อน · %ชดเชยร้านใหม่รวมกันต้อง = 100% (ไม่งั้น 422)
-- optimistic concurrency: mutation ทุกชุดต้องส่ง versionNo ล่าสุด; ไม่ตรงคืน 409 STALE_VERSION
UPDATE sgi_compensation_documents SET version_no = version_no + 1, updated_at = $1 /* now */, updated_by = $2 /* empId */
WHERE doc_no = $3 /* docNo */ AND version_no = $4 /* versionNo */;
-- ▲ :amount **ไม่ใช้คำนวณ client** — BE คิดจาก :pct × total_compensation_amount ของหัวเอกสาร
-- RETURNING ไว้ตรวจว่าโดนแถวจริง · ไม่โดน = 422 NEW_STORE_NOT_IN_DOCUMENT (ห้ามเงียบ)
-- ตารางนี้ไม่มี INSERT/DELETE โดยตั้งใจ — จำนวนร้านเปิดใหม่มาจาก ALLMAP ผ่าน Job 9
UPDATE sgi_document_new_stores SET compensate_percent = $5 /* pct */, compensation_amount = $6 /* amount */
WHERE new_store_code = $7 /* newStoreCode */ AND doc_no = $3 /* docNo */ RETURNING new_store_code;
UPDATE sgi_document_competitors SET impact_date = $8 /* date */ WHERE id = $9 /* competitorId */ AND doc_no = $3 /* docNo */;
UPDATE sgi_document_external_factors SET date_from = $10 /* from */, date_to = $11 /* to */ WHERE id = $12 /* factorId */ AND doc_no = $3
/* docNo */;

-- แถวที่ผู้ใช้ "เพิ่มเอง" (element ที่ไม่มี id) ต้อง INSERT ไม่ใช่ UPDATE — ของเดิมมีแต่ UPDATE/DELETE
-- brandCode ต้องมีอยู่ใน master sgi_competitors จริง (ไม่ใช่ free text)
-- ● ตรวจด้วย SELECT จากตาราง master — **ห้าม hardcode ช่วง 01-11** (แก้ 2026-09-23)
-- หน้าจอ master เพิ่มแบรนด์ใหม่ได้ รหัสที่ 12 ขึ้นไปเคยถูกปฏิเสธทั้งที่ถูกต้อง
-- competitorStoreCode คือรหัสสาขา (ว่างได้) · source_system = USER
INSERT INTO sgi_document_competitors (doc_no, competitor_store_code, brand_code, impact_date, source_system)
VALUES ($3 /* docNo */, $13 /* competitorStoreCode */, $14 /* brandCode */, $15 /* impactDate */, 'USER')
ON CONFLICT ON CONSTRAINT uq_doc_competitor DO UPDATE SET impact_date = EXCLUDED.impact_date;

INSERT INTO sgi_document_external_factors (doc_no, factor_code, date_from, date_to)
VALUES ($3 /* docNo */, $16 /* factorCode */, $17 /* dateFrom */, $18 /* dateTo */)
ON CONFLICT ON CONSTRAINT uq_doc_factor DO UPDATE SET date_to = EXCLUDED.date_to;

-- ลบรายการที่ผู้ใช้เอาออก (ปุ่ม "ลบที่เลือก") ส่งอาร์เรย์ชุดใหม่มาแทนทั้งชุด)
DELETE FROM sgi_document_competitors WHERE doc_no = $3 /* docNo */ AND id NOT IN ($19 /* keepCompetitorIds */);
DELETE FROM sgi_document_external_factors WHERE doc_no = $3 /* docNo */ AND id NOT IN ($20 /* keepFactorIds */);
```

10.3 Index / Constraint ที่ควรมี (ข้อเสนอ)

Table	DDL ที่เสนอ	ที่มา / หมายเหตุ
sgi_fgi_impact_compensations	CREATE INDEX idx_sgi_fgi_impact_compensations_impacted_store_code_compensate ON sgi_fgi_impact_compensations (impacted_store_code, compensate_month, compensate_seq);	ข้อเสนอ — อนุมานจากคอลัมน์ที่ปรากฏใน WHERE/JOIN ของ SQL ด้านบน ต้องวัด EXPLAIN ก่อนใช้จริง

ทั้งหมดเป็น ข้อเสนอ ไม่ใช่ข้อกำหนดจาก ข้อกำหนดทางธุรกิจ — ให้ตรวจกับ `EXPLAIN ANALYZE` บนข้อมูลจริง และรวมเข้าไฟล์ `sql/deploy-sgi-*.sql` แบบ idempotent (`CREATE INDEX IF NOT EXISTS`) ตาม pattern ที่ทีมใช้อยู่

11. Processing Flow

Step	Description
1	Validate required fields
2	Check duplicate store/month
3	Generate docNo
4	Insert sgi_compensation_documents

Step	Description
5	Open workflow task
6	Save section updates in transaction

12. Acceptance Criteria

- duplicate business key returns 409
- docNo format YYYY/xxxxx
- compensatePercent sum=100
- requestId trace does not replace business duplicate guard

13. Developer Test Checklist

No	Test
1	create success
2	create duplicate
3	update allocation invalid
4	permission denied section

14. Unit Test Scope

8 ชั่วโมง (30% ของ implementation 24 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`docNo`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required when opening existing document · รูปแบบ: YYYY/xxxxx
`storeCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: numeric length = 5 · รูปแบบ: string 5 digits
`amount`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: >= 0 · รูปแบบ: number, 2 decimals
`percent`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: 0-100 · รูปแบบ: number, 2 decimals
`sourceSystem`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: ALLMAP / USER · รูปแบบ: enum
`date`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: valid date · รูปแบบ: DD/MM/YYYY
`attachment`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: <= 5 MB · รูปแบบ: file
`source`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required · รูปแบบ: MANUAL|FS
business rule	logic	duplicate business key returns 409
business rule	logic	docNo format YYYY/xxxxx
business rule	logic	compensatePercent sum=100
business rule	logic	requestId trace does not replace business duplicate guard

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`POST /api/v1/sgi/document`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`PUT /api/v1/sgi/document/{docNo}`	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
`sgi_compensation_documents`, `workflow_transaction` / `workflow_approver` (`@srm/glb-workflow`), `sgi_document_new_stores`	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
service	error mapping	แปลง error ของ repository/lib เป็น error code ตามสัญญากลาง (LLDD-BE-API-Common-Contracts.pdf)

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด